Tema 04 — Programación orientada a objetos en PHP

IES Ntra. Sra. de los Remedios (Ubrique) · 2º DAW · Desarrollo Web en Entorno Servidor · Curso 2026/2027

> 🎯 **Idea central del tema:** hasta ahora tu código eran **variables y funciones sueltas**. Con la programación orientada a objetos (POO) **agrupas datos y comportamiento en una misma pieza (el objeto)**, ocultas los detalles internos y reutilizas código mediante la herencia. Es la base de cualquier framework moderno (Laravel, Symfony…).

Índice

- Introducción y objetivos
- Marco normativo y currículo
- Mapa del tema
- Convenciones y leyenda de recuadros

1. Metodologías de programación en PHP
2. Clases
3. Objetos
4. Constructor y destructor
5. Visibilidad: public, private y protected
6. Getters y setters (encapsulación)
7. Herencia
8. Elementos estáticos y constantes de clase
9. Más herramientas de POO (ampliación)
10. Novedades de PHP 8 en POO
11. Proyecto integrador: «Biblioteca»
12. Errores típicos y cómo depurarlos
13. Actividades y autoevaluación

- Resumen del tema

Introducción

La **programación orientada a objetos** es una metodología muy extendida en la que los sistemas se modelan mediante **clases**, que reúnen un conjunto de datos y de funcionalidades. Las clases son **definiciones**; a partir de ellas se crean **objetos**, que son ejemplares concretos de una clase y disponen de los datos y funcionalidades que ella define.

La POO permite concebir los programas de forma intuitiva y cercana a la realidad. Hoy es el paradigma dominante: PHP la implementa de forma completa, y lenguajes como Java o los de la plataforma .NET están orientados a objetos por completo. Dominarla es imprescindible para desarrollar en casi cualquier lenguaje moderno.

Un **objeto** es una entidad independiente con sus propios **datos** (atributos) y su propia **programación** (métodos). Una ventana, un menú o una carpeta de archivos pueden verse como objetos. El motor de un coche también: sus atributos describen sus características físicas y sus métodos describen su funcionamiento y su relación con otras partes del coche (que a su vez son objetos).

La idea que renovó la programación es la unión de datos y funciones en una misma entidad, llamada **encapsulación**. Por ejemplo, un objeto *página* contiene sus dimensiones (ancho, alto), su color, el estilo de borde... (atributos), y encapsulados con ellos están los métodos para cambiar el tamaño, el color o mostrar texto. Cuando otra parte del programa necesita que la página cambie de color, le **envía un mensaje** (llama a un método). A quien envía el mensaje no le interesa **cómo** lo hace la página ni qué estructuras de datos usa: eso queda **oculto**.

En este tema se explica la sintaxis de PHP para trabajar con objetos, sin entrar en toda la teoría general de la POO.

Objetivos

- Distinguir las metodologías de programación lineal, estructurada y orientada a objetos.
- Definir clases con atributos y métodos, y crear objetos a partir de ellas.
- Usar constructores, destructores y la referencia `$this`.
- Aplicar la encapsulación con modificadores de visibilidad, getters y setters.
- Crear jerarquías de clases mediante herencia y reutilizar el constructor del padre.
- Utilizar atributos, métodos y constantes estáticos.
- Organizar las clases en ficheros independientes y cargarlas con `require_once` o autocarga.

Marco normativo y currículo

Este tema corresponde al módulo profesional **0613 «Desarrollo web en entorno servidor»** del ciclo de **Técnico Superior en Desarrollo de Aplicaciones Web**, regulado por el **Real Decreto 686/2010, de 20 de mayo**, con la redacción dada por el **Real Decreto 405/2023**. Cada comunidad autónoma concreta el currículo con su propia orden (en Andalucía, la orden del ciclo DAW).

El tema trabaja el resultado de aprendizaje **RA4** y refuerza criterios de **RA2** y **RA3**:

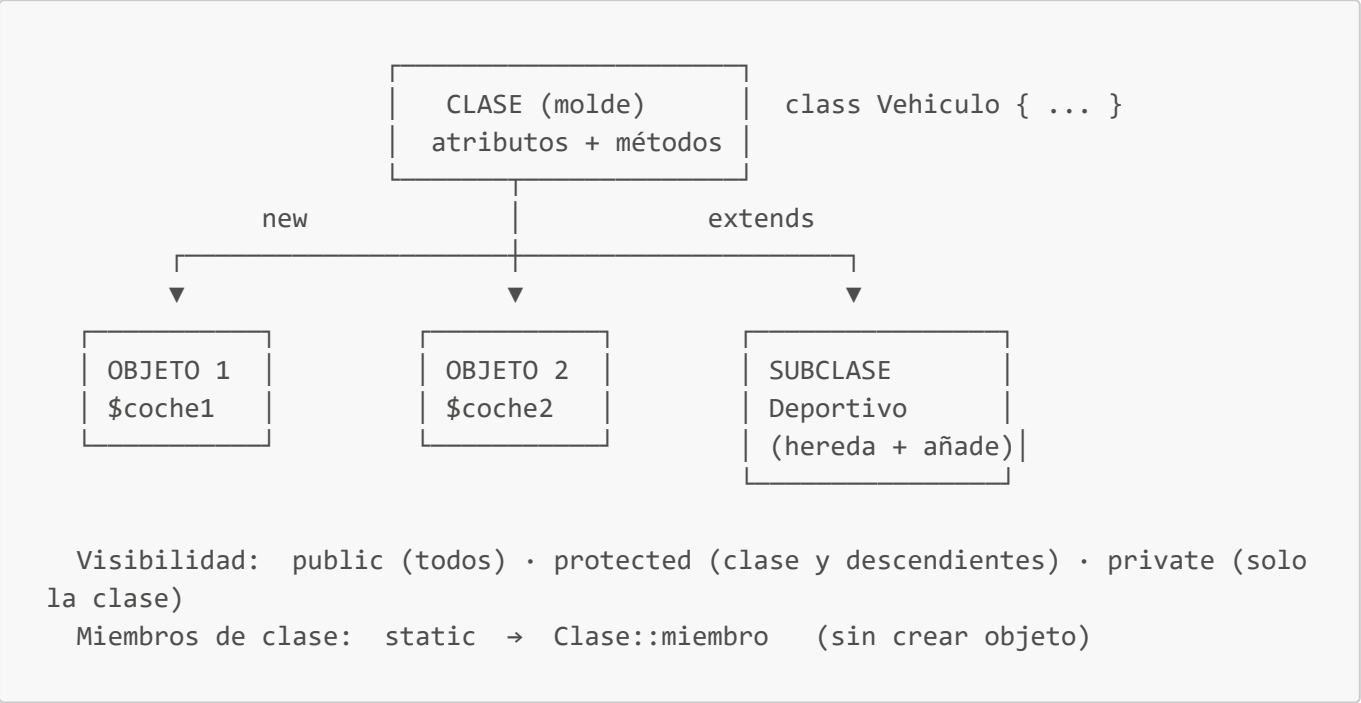
> **RA4.** Desarrolla aplicaciones web embebidas en lenguajes de marcas analizando e incorporando funcionalidades según especificaciones.

Criterio de evaluación	Contenido del tema	Apartado
RA4.g) Se han aplicado los principios de la programación orientada a objetos.	Clases, objetos, encapsulación, herencia, polimorfismo, abstracción	Todo el tema
RA4.h) Se ha probado y documentado el código.	Pruebas del proyecto integrador, comentarios y documentación (PHPDoc)	11 y 12
RA3.d) Se han creado y utilizado funciones.	Métodos como funciones asociadas a una clase	2 y 4
RA2.f) Se han utilizado directivas para modificar el comportamiento predeterminado.	`require_once` para cargar las clases	3.4

Criterio de evaluación	Contenido del tema	Apartado
RA2.g) Se han utilizado los distintos tipos de variables y operadores disponibles.	Tipado de atributos, parámetros y retornos	5 y 10

> **Nota:** los criterios están citados de forma resumida y su numeración puede variar según la versión del currículo. Deben contrastarse con la versión consolidada vigente en el BOE y con la orden autonómica aplicable al centro.

Mapa del tema



Convenciones del tema

Icono	Significado
💡	Truco o idea clave
⚠️	Error frecuente o peligro
🔒	Seguridad
🆕	Novedad de PHP 8.x
🧪	Pruébalo tú
🧠	Para pensar
📜	Normativa o currículo

Los bloques de código con la etiqueta `php` son ejemplos ejecutables; los marcados como `text` son esquemas o salidas. Todos los ejemplos usan el estilo recomendado por **PSR-12** (nombres de clase en `PascalCase`, métodos en `camelCase`, una clase por fichero).

> 📜 **Sobre los nombres de clase.** En los apuntes de cursos anteriores se usaba el prefijo `Class_` (por ejemplo `Class_persona`). **No es una convención de PHP**: el estándar PSR-1 pide nombres en

> `PascalCase` sin prefijos (`Persona`, `Vehiculo`). En este curso usaremos `Persona`, `Vehiculo`, etc.

---

1. Metodologías de programación en PHP

PHP permite programar con tres metodologías, que pueden combinarse:

Metodología	Descripción
Programación lineal	Todo el código se escribe como instrucciones PHP alternadas con el HTML de la página.
Programación estructurada	Se definen **funciones** que agrupan tareas y se llaman desde la página; pueden estar en el mismo fichero o en una librería aparte.
Programación orientada a objetos	Se definen **clases** y se crean **objetos** a partir de ellas. Es el objetivo de este tema.

> 💡 Las tres no se excluyen: una aplicación real usa POO para su lógica, funciones auxiliares y HTML en las vistas.

Conceptos clave de la POO

Concepto	Significado
Abstracción	Quedarse solo con las características relevantes de lo que se modela (de un gato: nombre, edad; no el número de pelos).
Encapsulación	Agrupar datos y métodos, y **ocultar** los detalles internos.
Herencia	Crear clases nuevas a partir de otras existentes, reutilizando su código.
Polimorfismo	Objetos de distintas clases responden al mismo mensaje (método) cada uno a su manera.

---

2. Clases

Una **clase** es un «molde» a partir del cual se crean objetos. Al definirla se decide qué **propiedades** y **métodos** tendrán sus objetos; a esto se le llama **abstracción**.

Una clase se compone de:

- **Atributos** (o propiedades): las características del objeto; son variables con un valor inicial.
- **Métodos**: las acciones que sabe hacer. Se nombran normalmente con **verbos** (un gato: `correr()`, `morder()`, `comer()`, `saltar()`). Son, en esencia, las funciones de la clase.

2.1. Declaración de una clase

```
class NombreClase
{
    [atributos]
```

```
    [métodos]
}
```

La palabra reservada es `class`, seguida del nombre y, entre llaves, los atributos y métodos. Conviene elegir un nombre **descriptivo** de lo que representa, en singular y con la inicial en mayúscula (`Persona`, `Vehiculo`).

Ejemplo: clase `Persona`

```php
<?php

class Persona
{
    private ?string $nombre;

    public function __construct()
    {
        $this->nombre = null;
    }

    public function getNombre(): ?string
    {
        return $this->nombre;
    }

    public function setNombre(string $nombre): void
    {
        $this->nombre = $nombre;
    }

    public function imprimir(): void
    {
        echo $this->nombre;
        echo '<br>';
    }
}
```

Ejemplo: clase `Vehiculo`

```php
<?php

class Vehiculo
{
    private ?string $matricula;
    private float $velocidad;

    public function __construct()
    {
        $this->matricula = null;
```

```
        $this->velocidad = 0;
    }

    public function getMatricula(): ?string
    {
        return $this->matricula;
    }

    public function getVelocidad(): float
    {
        return $this->velocidad;
    }

    public function setMatricula(string $matricula): void
    {
        $this->matricula = $matricula;
    }

    public function setVelocidad(float $velocidad): void
    {
        $this->velocidad = $velocidad;
    }

    public function aumentarVelocidad(): void
    {
        $this->velocidad++;
    }

    public function disminuirVelocidad(): void
    {
        $this->velocidad--;
    }

    public function parar(): void
    {
        $this->velocidad = 0;
    }
}
```

> ⚠️ **Error frecuente:** PHP distingue mayúsculas en nombres de variables y atributos, **no** en los nombres de clases y funciones. Aun así, escribe siempre `Persona`, nunca `persona` ni `PERSONA`.

2.2. La variable `$this`

Dentro de los métodos, para referirse a los atributos del propio objeto se usa `$this->` seguido del nombre del atributo **sin** el signo `$`.

`$this` significa «**este objeto**», la instancia que está ejecutando el método. Decir `$this->velocidad = 5` equivale a «asigna 5 al atributo `velocidad` de este objeto».

```php
<?php

class Contador
{
    private int $valor = 0;

    public function incrementar(): void
    {
        $this->valor++;      // atributo de ESTE objeto
    }
}
```

> ⚠️ Olvidar `$this->` es el error más típico: `$valor++` dentro del método crea una variable local nueva y no toca el atributo.

---

3. Objetos

Una vez definida la clase, se crean **objetos** a partir de ella. Cada objeto creado se denomina **instancia**.

3.1. Crear un objeto

```php
$objeto = new NombreClase();
```

El operador `new` reserva memoria para el objeto y ejecuta su método constructor `__construct()` (apartado 4).

3.2. Usar un objeto

Para acceder a un método o atributo se escribe el objeto, el operador flecha `->` y el miembro. Para poder hacerlo desde fuera, el miembro debe ser **público** (`public`).

```php
<?php

require_once 'Persona.php';

// Crea el objeto
$per1 = new Persona();

// Asigna el nombre mediante el método setNombre
$per1->setNombre('Juan');

// Muestra el nombre
echo $per1->getNombre();

// ERROR: nombre es privado y no puede usarse fuera de la clase
// echo $per1->nombre;
```

`$per1` parece una variable, pero es una **instancia** de `Persona`: posee todos los atributos de la clase y puede ejecutar todos sus métodos públicos. Se pueden crear tantos objetos como se necesiten, y **cada uno tiene sus propios datos**:

```php
<?php

$per2 = new Persona();
$per2->setNombre('Ana');
$per2->imprimir();      // Ana
```

3.3. Ejemplo: clases y objetos

```php
<?php

require_once 'Persona.php';
require_once 'Vehiculo.php';

$per1 = new Persona();
$per1->setNombre('Juan');
$per1->imprimir();              // Juan

$per2 = new Persona();
$per2->setNombre('Ana');
$per2->imprimir();              // Ana

$coche1 = new Vehiculo();
$coche1->setMatricula('0101 BFH');
$coche1->setVelocidad(10);
$coche1->aumentarVelocidad();
echo $coche1->getVelocidad();   // 11
```

```php
$coche2 = new Vehiculo();        // independiente de $coche1
echo $coche2->getVelocidad();    // 0
```

3.4. Una clase, un fichero

La buena práctica es guardar **cada clase en su propio fichero**, con el mismo nombre que la clase (`Persona.php`, `Vehiculo.php`) y cargarla con `require_once`, que evita cargarla dos veces. En los ficheros que solo contienen clases se **omite** la etiqueta de cierre `?>`.

```
proyecto/
├── index.php            ← usa las clases
└── clases/
    ├── Persona.php
    └── Vehiculo.php
```

```php
<?php

require_once __DIR__ . '/clases/Persona.php';
require_once __DIR__ . '/clases/Vehiculo.php';
```

> 💡 `__DIR__` es la carpeta del fichero actual: hace que la ruta funcione se ejecute desde donde se ejecute. En aplicaciones grandes se sustituye por la **autocarga** (apartado 9.6).

3.5. Objetos y referencias

En PHP, una variable que contiene un objeto guarda un **identificador (referencia)** al objeto, no una copia. Asignar esa variable a otra **no duplica** el objeto; para copiarlo se usa `clone`.

```php
<?php

$a = new Persona();
$a->setNombre('Ana');

$b = $a;                 // $b apunta al MISMO objeto
$b->setNombre('Luis');
echo $a->getNombre();    // Luis

$c = clone $a;           // copia independiente
$c->setNombre('Eva');
echo $a->getNombre();    // Luis
```

---

4. Constructor y destructor

4.1. Método `__construct()`

PHP permite declarar **constructores**. Una clase que lo tenga lo invoca **automáticamente** en cada objeto nuevo, lo que lo hace idóneo para inicializar el objeto antes de usarlo.

Características:

- Es el primer método que se ejecuta al crear el objeto.
- Se llama automáticamente con `new`.
- **No devuelve ningún dato.**
- Puede recibir **parámetros**, normalmente para inicializar atributos.
- Es **opcional**.

```
public function __construct([parámetros])
{
    [algoritmo]
}
```

Ejemplo: clase `Cliente`

```php
<?php

class Cliente
{
    private ?string $nombre;
    private ?int $numero;
    private array $peliculasAlquiladas;

    public function __construct(
        ?string $nombre = null,
        ?int $numero = null,
        array $peliculasAlquiladas = []
    ) {
        $this->nombre = $nombre;
        $this->numero = $numero;
        $this->peliculasAlquiladas = $peliculasAlquiladas;
    }

    public function dameNumero(): ?int
    {
        return $this->numero;
    }
}

// Instanciamos dos clientes
$cliente1 = new Cliente('Pepe', 1);
$cliente2 = new Cliente('Roberto', 564);

echo 'El identificador del cliente 1 es: ' . $cliente1->dameNumero() . '<br>';
echo 'El identificador del cliente 2 es: ' . $cliente2->dameNumero() . '<br>';
```

```
El identificador del cliente 1 es: 1
El identificador del cliente 2 es: 564
```

> 🧠 **Para pensar:** PHP **no admite sobrecarga** de constructores (no puede haber dos `__construct` con distintos parámetros). Para ofrecer varias formas de crear un objeto se usan parámetros opcionales o métodos estáticos de fábrica (apartado 8).

4.2. Método `__destruct()`

El **destructor** realiza las tareas necesarias cuando un objeto deja de existir. Cuando ya no hay ninguna variable que lo referencie, el objeto se elimina de memoria y justo antes se llama a `__destruct()`.

Características:

- Su objetivo principal es **liberar recursos** que pidió el objeto (conexión a base de datos, ficheros abiertos, imágenes dinámicas...).
- Es el último método que se ejecuta.
- Se ejecuta **automáticamente**.
- Debe llamarse exactamente `__destruct` y **no devuelve datos**.
- Es **opcional** y menos habitual que el constructor, porque PHP libera la memoria por sí solo.

El objeto se destruye cuando: se hace `unset()` de la última referencia, se asigna `null` a la variable, la variable sale de su ámbito o **termina el script**.

```php
<?php

class Persona
{
    public function __construct(
        public ?string $nombre = null,
        public ?string $apellido = null,
        public ?int $edad = null
    ) {
    }

    public function __destruct()
    {
        echo 'Objeto destruido';
    }

    public function saludar(): string
    {
        return 'Hola, soy ' . $this->nombre . ' ' . $this->apellido
            . ' y tengo ' . $this->edad . ' años. ';
    }
}

$persona = new Persona('Fernando', 'Gaitán', 26);
echo $persona->saludar();
```

```php
// Destruimos el objeto: __destruct() mostrará «Objeto destruido»
unset($persona);
```

```
Hola, soy Fernando Gaitán y tengo 26 años. Objeto destruido
```

---

5. Visibilidad: public, private y protected

Cuando se crea un objeto, los atributos de su clase se reservan en memoria y pueden modificarse mediante los métodos. Los atributos pueden ser de cualquiera de los tipos de PHP ya conocidos. Según su **visibilidad** hay tres niveles, aplicables también a los métodos:

Modificador	Se puede acceder desde...
`public`	Cualquier sitio (dentro y fuera de la clase).
`protected`	La propia clase y sus **clases hijas**.
`private`	**Solo** la propia clase.

> ⚠️ Si se omite el modificador en un método, es `public`. En los atributos **no se puede omitir**: se declara siempre su visibilidad (o `var`, obsoleto). Los apuntes de otros cursos que escriben `$velocidad;` a secas no funcionan en PHP moderno.

5.1. Atributos públicos

Permiten interactuar con ellos **desde el exterior**.

```php
<?php

class Coche
{
    public int $velocidad;

    public function __construct()
    {
        $this->velocidad = 0;
    }

    public function aumentarVelocidad(): void
    {
        $this->velocidad++;
    }

    public function parar(): void
    {
        $this->velocidad = 0;
    }
```

```
}

$coche = new Coche();
$coche->velocidad = 1000;       // velocidad = 1000 (¡sin control!)
```

> ⚠️ Un atributo público viola la **encapsulación**: cualquiera puede asignarle un valor absurdo (`-50`, `'hola'`...) sin que la clase lo controle.

5.2. Atributos privados

Según la filosofía de la POO, los atributos deben permanecer **ocultos** al exterior; por eso se usa `private`. Solo los métodos de la propia clase pueden acceder a ellos. Se permite un acceso controlado mediante **getters y setters** (apartado 6). **Las clases hijas tampoco** pueden acceder a los atributos `private`.

```php
<?php

class Coche
{
    private int $velocidad;

    public function __construct()
    {
        $this->velocidad = 0;
    }

    public function setVelocidad(int $velocidad): void
    {
        $this->velocidad = $velocidad;
    }

    public function getVelocidad(): int
    {
        return $this->velocidad;
    }
}

$coche = new Coche();
$coche->setVelocidad(1000);
echo $coche->getVelocidad();    // 1000

echo $coche->velocidad;         // Error: no se puede acceder al atributo privado
```

```
Fatal error: Uncaught Error: Cannot access private property Coche::$velocidad
```

5.3. Atributos protegidos

`protected` permite acceder al atributo desde la clase que lo define **y desde cualquiera de sus clases hijas**. La clase hija hereda los miembros `public` y `protected` de la clase madre.

Veamos qué ocurre con `private`:

```php
<?php

class Coche
{
    private int $velocidad = 0;

    public function setVelocidad(int $velocidad): void
    {
        $this->velocidad = $velocidad;
    }
}

class Deportivo extends Coche
{
    public function aumentarVelocidad(): void
    {
        // Error: el atributo es privado de Coche; Deportivo no lo ve
        $this->velocidad = $this->velocidad * 2;
    }
}

$deportivo = new Deportivo();
$deportivo->setVelocidad(100);
$deportivo->aumentarVelocidad();    // Warning: Undefined property
Deportivo::$velocidad
```

Los métodos heredados de `Coche` sí acceden al atributo privado (porque se ejecutan «dentro» de `Coche`), pero **los métodos nuevos de la subclase no**. Para permitirlo existe `protected`:

```php
<?php

class Coche
{
    protected int $velocidad = 0;

    public function setVelocidad(int $velocidad): void
    {
        $this->velocidad = $velocidad;
    }

    public function getVelocidad(): int
    {
        return $this->velocidad;
    }
}
```

```php
class Deportivo extends Coche
{
    public function aumentarVelocidad(): void
    {
        $this->velocidad = $this->velocidad * 2;
    }
}

$deportivo = new Deportivo();
$deportivo->setVelocidad(100);
$deportivo->aumentarVelocidad();
echo $deportivo->getVelocidad();    // 200
```

> 💡 **Buena práctica:** declara los atributos siempre como `private`. Si la clase está pensada para ser padre y quieres delegar el control del atributo a las hijas, usa `protected`. Los métodos que forman la interfaz de la clase se declaran `public`; los auxiliares internos, `private` o `protected`.

5.4. Resumen de visibilidad

Desde...	`public`	`protected`	`private`
La propia clase	✅	✅	✅
Una clase hija	✅	✅	❌
Fuera de la clase	✅	❌	❌

---

6. Getters y setters (encapsulación)

Declarar atributos `public` violaría una regla básica de la POO: la **encapsulación**. Los datos internos solo deberían accederse mediante métodos de la propia clase. Desde fuera no se debería asignar ni leer un atributo directamente; para eso existen los **getters** y los **setters**.

- **Getter**: no recibe parámetros y devuelve con `return` el valor del atributo.
- **Setter**: recibe **un parámetro** (el nuevo valor) y se lo asigna al atributo. No devuelve nada.

```php
public function getNumber(): int      { return $this->number; }
public function setNumber(int $value): void { $this->number = $value; }
```

La gran ventaja del setter es que **puede validar** el valor antes de aceptarlo:

Ejemplo completo: `Cuadrado.php`

```php
<?php

class Cuadrado
{
```

```
    private float $lado = 0;

    public function setLado(float $valor): void
    {
        if ($valor < 0) {
            throw new InvalidArgumentException('El lado no puede ser negativo');
        }
        $this->lado = $valor;
    }

    public function getLado(): float
    {
        return $this->lado;
    }

    public function area(): float
    {
        return $this->lado * $this->lado;
    }
}
```

Y su uso desde `index.php`:

```
<?php

require_once __DIR__ . '/Cuadrado.php';

$instancia = new Cuadrado();
$instancia->setLado(12);
echo $instancia->area();            // 144
```

> 💡 No hay que crear getters y setters «por sistema» para todos los atributos. Crea solo los que necesite el resto del programa; un atributo sin setter es de **solo lectura** desde fuera.

---

7. Herencia

La **herencia** permite crear clases nuevas a partir de otras existentes. La clase nueva hereda todos los atributos y métodos accesibles (`public` y `protected`) de su **superclase** o **clase padre**, y además puede incorporar los suyos propios.

> En PHP, a diferencia de C++, una clase solo puede derivar de **una única clase**: **no existe herencia múltiple**. Para compartir comportamiento entre varias clases se usan *interfaces* y *traits* (apartado 9).

7.1. Superclase o clase padre

Clase de la que desciende otra. Sus clases hijas incorporan automáticamente sus atributos y métodos.

7.2. Subclase

Clase que desciende de otra. Es una **especialización** de su superclase y admite nuevos atributos y métodos que aumentan esa especialización.

```
class ClaseHija extends ClasePadre
{
    [atributos]

    [métodos]
}
```

```
        Vehiculo            ← superclase
        ▲      ▲
        |      |
 Deportivo   Camion          ← subclases
```

En el ejemplo, a partir de `Vehiculo` creamos la subclase `Deportivo` con nuevos atributos y métodos:

```php
<?php

class Vehiculo
{
    protected float $velocidad = 0;

    public function aumentarVelocidad(): void
    {
        $this->velocidad++;
    }

    public function getVelocidad(): float
    {
        return $this->velocidad;
    }
}

class Deportivo extends Vehiculo
{
    private bool $aleronDeportivo = true;
    private bool $cristalesTintados = true;
    private int $musica = 0;

    public function ponerMusica(): void
    {
        $this->musica = 1;
    }

    public function apagarMusica(): void
    {
        $this->musica = 0;
```

```
    }
}

$deportivo = new Deportivo();
$deportivo->aumentarVelocidad();    // velocidad = 1 (método heredado)
$deportivo->aumentarVelocidad();    // velocidad = 2
$deportivo->ponerMusica();          // música = 1 (método propio)
```

7.3. Herencia y constructores

El constructor del padre no se llama implícitamente si la clase hija define el suyo. Para ejecutarlo hay que invocar `parent::__construct()` desde el constructor hijo. Si la hija **no** define constructor, hereda el del padre como un método normal.

Caso 1: la hija no define constructor

```
<?php

class Producto
{
    public function __construct(
        protected ?int $id = null,
        protected ?string $titulo = null,
        protected ?float $precio = null,
        protected ?string $nombreAutor = null,
        protected ?string $apellidosAutor = null
    ) {
    }

    public function getNombreAutor(): ?string
    {
        return $this->nombreAutor;
    }

    public function getApellidosAutor(): ?string
    {
        return $this->apellidosAutor;
    }

    public function getTitulo(): ?string
    {
        return $this->titulo;
    }

    public function getPrecio(): ?float
    {
        return $this->precio;
    }
}

class Libro extends Producto
```

```
{
    public function getResumen(): string
    {
        return 'Título: ' . $this->getTitulo()
            . ', Precio: ' . $this->getPrecio()
            . ', Autor: ' . $this->getNombreAutor() . ' ' . $this-
>getApellidosAutor();
    }
}
```

Como `Libro` no define constructor, se hereda el de `Producto`; al crear un libro hay que pasar los argumentos que pide el constructor base (si no se pasan, los atributos valen `null`):

```
<?php

$libro1 = new Libro(1, 'Título', 20, 'Autor', 'Apellido1 Apellido2');
echo $libro1->getResumen();
```

```
Título: Título, Precio: 20, Autor: Autor Apellido1 Apellido2
```

Caso 2: la hija define su propio constructor

```
<?php

class Libro extends Producto
{
    private ?int $numPaginas;

    public function __construct(
        ?int $id = null,
        ?string $titulo = null,
        ?float $precio = null,
        ?string $nombreAutor = null,
        ?string $apellidosAutor = null,
        ?int $numPaginas = null
    ) {
        parent::__construct($id, $titulo, $precio, $nombreAutor, $apellidosAutor);
        $this->numPaginas = $numPaginas;
    }

    public function getNumPaginas(): ?int
    {
        return $this->numPaginas;
    }

    public function getResumen(): string
    {
```

```
        return 'Título: ' . $this->getTitulo()
            . ', Precio: ' . $this->getPrecio()
            . ', Autor: ' . $this->getNombreAutor() . ' ' . $this-
>getApellidosAutor()
            . ', Núm. páginas: ' . $this->getNumPaginas();
    }
}
```

El constructor de `Libro` es un **añadido** al del padre: primero llama a `parent::__construct()` con los argumentos que este necesita y después añade su propia inicialización.

```php
<?php

$libro1 = new Libro(1, 'Título', 20, 'Autor', 'Apellido1 Apellido2', 440);
echo $libro1->getResumen();
```

```
Título: Título, Precio: 20, Autor: Autor Apellido1 Apellido2, Núm. páginas: 440
```

> ⚠️ **Errores del material antiguo corregidos aquí:** el resumen llamaba a un método `getAutor()` que no existía en `Producto`, y faltaba el `;` tras algunos `return`. Un método que se invoca debe estar definido en la clase o en alguno de sus ancestros.

7.4. Sobrescritura de métodos (override)

Una subclase puede **redefinir** un método heredado, con el mismo nombre y compatible en parámetros. Desde él se puede llamar a la versión del padre con `parent::`.

```php
<?php

class Animal
{
    public function hablar(): string
    {
        return '...';
    }
}

class Perro extends Animal
{
    public function hablar(): string
    {
        return 'Guau';
    }
}

class Gato extends Animal
```

```
{
    public function hablar(): string
    {
        return 'Miau';
    }
}

// Polimorfismo: mismo mensaje, respuesta distinta según la clase
foreach ([new Perro(), new Gato(), new Animal()] as $animal) {
    echo $animal->hablar() . '<br>';
}
```

```
Guau
Miau
...
```

7.5. Impedir la herencia o la sobrescritura: `final`

- `final class Clase` → no se puede heredar de ella.
- `final public function metodo()` → no se puede sobrescribir en las hijas.

---

8. Elementos estáticos y constantes de clase

Los atributos y métodos **estáticos** no necesitan un objeto para usarse: se emplean directamente con el operador de resolución de ámbito `::` (en PHP, también llamado *Paamayim Nekudotayim*, «dos veces dos puntos» en hebreo). Pertenecen a la **clase**, no a cada objeto; por eso se llaman «elementos de clase».

Se declaran con la palabra reservada `static`.

```
class NombreClase
{
    public static $atributo;

    public static function metodo() { ... }
}
```

Reglas:

- Un método estático **no puede usar** `$this` ni atributos normales, porque se puede invocar sin objeto. Sí puede usar atributos estáticos.
- Dentro de la clase se accede a ellos con `self::`; desde fuera, con `NombreClase::`.
- Un atributo o método estático **no se accede con** `->` desde un objeto.

```
<?php
```

```php
class Prueba
{
    public static string $nombre = 'José';

    public static function saludo(): void
    {
        echo 'Hola ' . self::$nombre . ', os saluda';
    }
}

$p = new Prueba();

// $p->saludo();            // incorrecto: no se usa -> con miembros estáticos
// echo $p->nombre;         // incorrecto

echo Prueba::$nombre;       // José         (correcto)
Prueba::saludo();           // Hola José, os saluda (correcto)
```

> ⚠️ Corrección respecto al material antiguo: el atributo estático se accede con el `$`: `Prueba::$nombre`, no `Prueba::nombre`.

Uso típico: contador de instancias

```php
<?php

class Usuario
{
    private static int $total = 0;

    public function __construct(private string $nombre)
    {
        self::$total++;
    }

    public static function getTotal(): int
    {
        return self::$total;
    }
}

new Usuario('Ana');
new Usuario('Luis');
echo Usuario::getTotal();   // 2
```

8.1. Constantes de clase

Valores que **no cambian**, asociados a la clase. Se declaran con `const` y se acceden con `::` (sin `$`).

```php
<?php

class Circulo
{
    public const PI = 3.14159;

    public static function area(float $radio): float
    {
        return self::PI * $radio ** 2;
    }
}

echo Circulo::PI;           // 3.14159
echo Circulo::area(2);      // 12.56636
```

8.2. `self` frente a `static`

`self::` hace referencia a la clase **donde está escrito el código**. `static::` (*late static binding*) hace referencia a la clase **que realmente se ha llamado**, lo cual importa con herencia. Para empezar, usa `self`.

---

9. Más herramientas de POO (ampliación)

Este apartado amplía el material original y completa los principios de la POO que exige el currículo (RA4.g).

9.1. Clases abstractas

Una **clase abstracta** no se puede instanciar: sirve de base común. Puede declarar **métodos abstractos** (sin cuerpo) que las hijas están **obligadas** a implementar.

```php
<?php

abstract class Figura
{
    abstract public function area(): float;

    public function describir(): string
    {
        return static::class . ' con área ' . round($this->area(), 2);
    }
}

class Rectangulo extends Figura
{
    public function __construct(private float $base, private float $altura)
    {
    }

    public function area(): float
    {
```

```
        return $this->base * $this->altura;
    }
}

// $f = new Figura();              // Error: no se puede instanciar
$r = new Rectangulo(3, 4);
echo $r->describir();              // Rectangulo con área 12
```

9.2. Interfaces

Una **interfaz** define un **contrato**: lista de métodos públicos que una clase debe ofrecer, sin implementación. Una clase puede implementar **varias** interfaces (es la alternativa a la herencia múltiple).

```
<?php

interface Imprimible
{
    public function imprimir(): void;
}

interface Guardable
{
    public function guardar(): bool;
}

class Factura implements Imprimible, Guardable
{
    public function imprimir(): void
    {
        echo 'Factura impresa';
    }

    public function guardar(): bool
    {
        return true;
    }
}
```

	Clase abstracta	**Interfaz**
Atributos	Sí	No (solo constantes)
Métodos con código	Sí	No
Herencia / implementación	Una sola (`extends`)	Varias (`implements`)
Úsala cuando…	Las clases comparten código y son «una especie de»	Clases distintas deben cumplir un mismo contrato

9.3. Traits

Un **trait** es un grupo de métodos y atributos que se puede «pegar» en varias clases para reutilizar código sin herencia.

```php
<?php

trait RegistraLog
{
    public function log(string $mensaje): void
    {
        echo '[' . date('H:i:s') . '] ' . $mensaje . '<br>';
    }
}

class Pedido
{
    use RegistraLog;
}

(new Pedido())->log('Pedido creado');
```

9.4. Métodos mágicos útiles

Los nombres que empiezan por `__` son métodos especiales que PHP invoca en situaciones concretas.

Método	Se invoca cuando…
`__construct()`	Se crea el objeto.
`__destruct()`	Se destruye el objeto.
`__toString()`	El objeto se usa como cadena (`echo $obj`).
`__get($n)` / `__set($n,$v)`	Se lee/escribe una propiedad inaccesible o inexistente.
`__call($n,$args)`	Se llama a un método inexistente o inaccesible.
`__clone()`	Se clona el objeto con `clone`.

```php
<?php

class Punto
{
    public function __construct(private int $x, private int $y)
    {
    }

    public function __toString(): string
    {
        return "($this->x, $this->y)";
```

```
    }
}

echo new Punto(3, 5);       // (3, 5)
```

9.5. Operador `instanceof` y comprobación de tipos

```
<?php

$d = new Deportivo();

var_dump($d instanceof Deportivo);      // true
var_dump($d instanceof Vehiculo);       // true (hereda de Vehiculo)
var_dump($d instanceof Camion);         // false

echo get_class($d);                     // Deportivo
echo get_parent_class($d);              // Vehiculo
```

También se pueden **tipar parámetros con clases**: una función que pide un `Vehiculo` acepta cualquier `Deportivo` (polimorfismo).

```
<?php

function acelerar(Vehiculo $v): void
{
    $v->aumentarVelocidad();
}

acelerar(new Deportivo());      // válido
```

9.6. Espacios de nombres y autocarga (visión general)

Cuando hay muchas clases, `require_once` en cada una resulta incómodo. La función `spl_autoload_register()` carga una clase **automáticamente** la primera vez que se usa.

```
<?php

spl_autoload_register(function (string $clase): void {
    require_once __DIR__ . '/clases/' . $clase . '.php';
});

$p = new Persona();     // carga clases/Persona.php sin require manual
```

Los **espacios de nombres** (`namespace App\Modelo;`) evitan choques de nombres entre clases de distintas librerías; junto con la autocarga de **Composer** (estándar PSR-4) son la base de los proyectos profesionales,

que se verán más adelante.

9.7. Documentar clases con PHPDoc (RA4.h)

```php
<?php

/**
 * Representa un vehículo con control de velocidad.
 */
class Vehiculo
{
    /**
     * Aumenta la velocidad en una unidad.
     *
     * @return void
     */
    public function aumentarVelocidad(): void
    {
        // ...
    }
}
```

---

10. Novedades de PHP 8 en POO

> 🆕 Resumen de funciones modernas que verás en código actual.

Novedad	Versión	Ejemplo
Promoción de propiedades en el constructor	8.0	`public function __construct(private string $nombre) {}`
Tipos de propiedad (tipado)	7.4	`private int $edad;`
Tipo de unión	8.0	`int|float $precio`
Operador `?->` (nullsafe)	8.0	`$a?->getB()?->getC()`
Enumeraciones (`enum`)	8.1	`enum Estado { case Activo; case Baja; }`
Propiedades `readonly`	8.1	`public readonly int $id;`
Clases `readonly`	8.2	`readonly class Punto {…}`
`new` en inicializadores	8.1	`function f(Log $l = new Log())`

Promoción de propiedades: declara y asigna los atributos directamente en la firma del constructor.

```php
<?php

// Forma clásica
class PersonaClasica
```

```
{
    private string $nombre;
    private int $edad;

    public function __construct(string $nombre, int $edad)
    {
        $this->nombre = $nombre;
        $this->edad = $edad;
    }
}

// Forma moderna (equivalente)
class PersonaModerna
{
    public function __construct(
        private string $nombre,
        private int $edad
    ) {
    }
}
```

`readonly`: el valor solo puede asignarse una vez (en el constructor); después es inmutable.

```
<?php

class Libro
{
    public function __construct(
        public readonly string $isbn,
        private string $titulo
    ) {
    }
}

$l = new Libro('978-84-1234-567-8', 'PHP moderno');
echo $l->isbn;
// $l->isbn = 'otro';    // Error: no se puede modificar una propiedad readonly
```

Enumeraciones

```
<?php

enum Estado: string
{
    case Disponible = 'D';
    case Prestado = 'P';

    public function etiqueta(): string
    {
```

```
        return match ($this) {
            self::Disponible => 'Disponible',
            self::Prestado => 'Prestado',
        };
    }
}

echo Estado::Prestado->etiqueta();      // Prestado
```

---

11. Proyecto integrador: «Biblioteca»

Aplicación en consola/web que gestiona libros de una biblioteca aplicando todo el tema.

Requisitos

1. Clase abstracta `Documento` con `titulo`, `codigo` y método abstracto `diasPrestamo(): int`.
2. Clases hijas `Libro` (con `numPaginas`, préstamo de 30 días) y `Revista` (con `numero`, préstamo de 7 días).
3. Todos los atributos `private` o `protected`, con getters y validación en los setters.
4. Enumeración `Estado` (`Disponible`, `Prestado`).
5. Clase `Biblioteca` con un array de documentos y métodos `añadir()`, `prestar($codigo)` y `listar()`.
6. Contador estático del total de documentos creados.
7. Cada clase en su fichero y cargadas con `spl_autoload_register`.

Estructura

```
biblioteca/
├── index.php
└── clases/
    ├── Documento.php
    ├── Libro.php
    ├── Revista.php
    ├── Estado.php
    └── Biblioteca.php
```

Esqueleto de partida

```
<?php

// clases/Documento.php
abstract class Documento
{
    private static int $total = 0;
    protected Estado $estado = Estado::Disponible;

    public function __construct(
        protected string $codigo,
```

```
            protected string $titulo
        ) {
            self::$total++;
        }

        abstract public function diasPrestamo(): int;

        public function getCodigo(): string
        {
            return $this->codigo;
        }

        public function getEstado(): Estado
        {
            return $this->estado;
        }

        public function prestar(): bool
        {
            if ($this->estado === Estado::Prestado) {
                return false;
            }
            $this->estado = Estado::Prestado;
            return true;
        }

        public static function getTotal(): int
        {
            return self::$total;
        }

        public function __toString(): string
        {
            return "[{$this->codigo}] {$this->titulo} ({$this->estado->name})";
        }
    }
```

Ejemplo de funcionamiento esperado

```
[L001] El Quijote (Disponible)
[R001] Hola Mundo PHP (Disponible)
Préstamo de L001: correcto (30 días)
Préstamo de L001: ya estaba prestado
Documentos creados: 2
```

---

12. Errores típicos y cómo depurarlos

Síntoma	Causa probable	Solución
`Undefined variable $velocidad` dentro de un método	Falta `$this->`	`$this->velocidad`
`Cannot access private property`	Acceso externo a un atributo `private`	Usar getter/setter
`Undefined property Hija::$x`	El atributo es `private` en el padre	Declararlo `protected`
`Call to undefined method`	Método inexistente o mal escrito	Revisar nombre y clase
`Class "X" not found`	Falta `require_once`/autocarga o nombre mal escrito	Cargar el fichero
`Cannot redeclare class`	Cargada dos veces con `require`	Usar `require_once`
`Using $this when not in object context`	`$this` en un método `static`	Quitar `static` o usar `self::`
`Non-static method cannot be called statically`	Llamar con `::` a un método de instancia	Crear el objeto y usar `->`
`Cannot instantiate abstract class`	`new` sobre una clase abstracta	Instanciar una hija concreta
El constructor del padre «no se ejecuta»	La hija define su constructor y no llama a `parent::__construct()`	Llamarlo explícitamente
Cambio un objeto y se modifica «otro»	Asignación por referencia	Usar `clone`

Herramientas útiles

```php
<?php

var_dump($objeto);              // estructura completa
print_r($objeto);               // volcado legible
echo get_class($objeto);        // clase del objeto
print_r(get_object_vars($objeto));
print_r(get_class_methods($objeto));
```

---

13. Actividades y autoevaluación

Actividades prácticas

1. **Persona.** Crea la clase `Persona` (nombre, apellidos, edad) con constructor, getters, setters y un método `saludar()`. Crea tres objetos y muéstralos. Impide edades negativas.

2. **Vehículos.** Implementa `Vehiculo` y `Deportivo` (hereda de `Vehiculo`). Añade a `Deportivo` el método `turbo()` que duplique la velocidad. Usa `protected`.
3. **Constructor del padre.** Crea `Producto` y `Libro` con `parent::__construct()`, y una nueva subclase `Dvd` con atributo `duracion`.
4. **Estáticos.** Añade a `Usuario` un contador de instancias y una constante `MAX_USUARIOS`; impide crear más de ese número.
5. **Polimorfismo.** Define la clase abstracta `Figura` y las hijas `Circulo`, `Rectangulo` y `Triangulo`. Recorre un array de figuras y muestra el área de cada una.
6. **Interfaz.** Crea la interfaz `Imprimible` e impleméntala en dos clases distintas.
7. **Proyecto.** Completa el proyecto integrador «Biblioteca» del apartado 11.

Autoevaluación

1. ¿Qué diferencia hay entre una clase y un objeto?
2. ¿Qué representa `$this`?
3. ¿Qué hace el operador `new`?
4. ¿Cuándo se ejecuta `__construct()` y qué debe devolver?
5. ¿Por qué se recomienda declarar los atributos `private`?
6. ¿Qué diferencia hay entre `private` y `protected`?
7. ¿Se llama automáticamente al constructor del padre cuando la hija define el suyo?
8. ¿Admite PHP herencia múltiple? ¿Qué alternativas hay?
9. ¿Cómo se accede a un atributo estático desde fuera de la clase? ¿Y desde dentro?
10. ¿Por qué un método estático no puede usar `$this`?
11. ¿Qué ocurre al ejecutar `$b = $a;` si `$a` es un objeto? ¿Cómo se obtiene una copia?
12. ¿En qué se diferencian una clase abstracta y una interfaz?

▶ Respuestas orientativas

Resumen del tema

- La **POO** modela el sistema con **clases** (moldes) y **objetos** (instancias), uniendo datos y comportamiento: **encapsulación**.
- Una clase se declara con `class`; un objeto se crea con `new` y se usa con `->`. `$this` es el propio objeto.
- El **constructor** (`__construct`) inicializa el objeto; el **destructor** (`__destruct`) libera recursos.
- La **visibilidad** (`public`, `protected`, `private`) controla el acceso; por defecto, atributos `private` con **getters y setters** que validan.
- La **herencia** (`extends`) reutiliza y especializa clases; solo hay un padre. El constructor del padre se invoca con `parent::__construct()`. Los métodos se pueden **sobrescribir** (polimorfismo).
- Los miembros `static` y las **constantes** pertenecen a la clase y se usan con `::` (`self::` dentro).
- **Clases abstractas, interfaces y traits** permiten diseñar jerarquías flexibles.
- PHP 8 aporta promoción de propiedades, `readonly`, `enum` y tipado estricto.
- Buenas prácticas: una clase por fichero, `PascalCase`, `require_once` o autocarga, y documentar con PHPDoc.